Bringing UNITY to Software and Hardware Design

Some **Rules** for the game

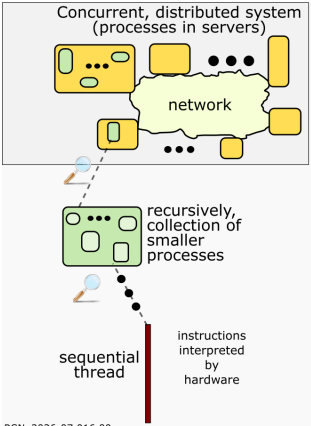
Rishiyur S. Nikhil
Co-founder and CTO
bluespec

July 17, 2026

@ Software Eng. Research in India (SERI) Update Mtng 2026
@ IIIT Hyderabad

PDF of slides
Source codes of examples
https://github.com/rsnikhil/Talk_Bring_UNITY_to_SW_HW_Design

My message in one slide, by means of an analogy



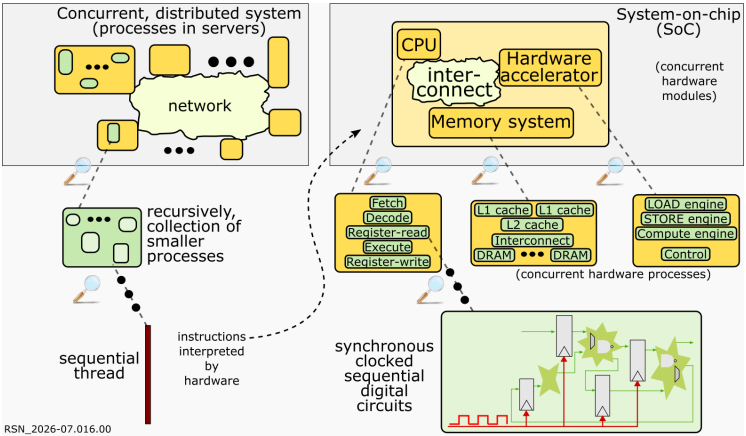
Physical span: office, rack, data center, inter-city, inter-continent

A long time ago, some eminent Computer Scientists recommended that *sequential programming be relegated only to the bottom-most layer of the software stack*.

They argued that *sequential languages do not scale upwards:* expressive power, modularity, composability, formalizability, verifiability.

- Concurrency: Atomic Rules, Guarded Commands [Dijkstra 1976], UNITY [Chandy+Misra 1988]
- Distribution: CSP (Communicating Sequential Processes) [Hoare 1978-1985]

My message in one slide, by means of an analogy



A long time ago, some eminent Computer Scientists recommended that *sequential programming be relegated only to the bottom-most layer of the software stack.*

They argued that *sequential languages do not scale upwards:* expressive power, modularity, composability, formalizability, verifiability.

- Concurrency: Atomic Rules, Guarded Commands [Dijkstra 1976], UNITY [Chandy+Misra 1988]
- Distribution: CSP (Communicating Sequential Processes) [Hoare 1978-1985]

Similarly, I argue: *for chip-scale hardware systems as well, traditional synchronous, clocked, sequential digital circuit design should be relegated to the bottom-most (most local) layer.*

The same concepts of distribution and concurrency are applicable at all but the lowest level. Instead of "synchronous", the watchwords for hardware design should be:

asynchronous, concurrent, distributed, overlapped, latency-tolerant, split-phase, message-passing, reorder-tolerant, elastic-pipelines, dataflow, ...

Why should SW engineers care?

- You can shape and teach HW design methodology, with programming models familiar to you (asynchronous, concurrent, distributed systems)!

  (... and not Verilog/SystemVerilog/VHDL !)

- You could roll your own hardware accelerators!
  - Hardware accelerators (computations implemented directly in hardware) are increasingly important, and unavoidable, for performance and energy reasons, by removing the software layer of interpretation.

    Added urgency due to end of Moore's Law, end of Dennard Scaling.
  - Custom hardware is increasingly accessible to the everyday software engineer (cheap, capacious, PCIe-attached FPGAs; on-CPU FPGAs).

"I found myself dismayed that people would consider themselves to be either hardware or software experts, but paid little heed to how joint advances in programming and architecture could lead to a synergistic outcome that might revolutionize computing practice."

Jack Dennis (1932-2026), Emeritus Professor of EECS, MIT

(Multics Operating System, Dataflow Architectures, Functional Programming Languages, ...)

(from a recollection in 2003)

"Hardware designers are from Earth, Software engineers are from Pluto (or vice versa) ...

... Not!"

(author)

My own background and interests, briefly

Functional Programming
Computer Architecture
High-Performance Computing Systems
Hardware design, languages and methodology

B.Tech EE IITK (1976)	Digital **hardware design**
M.S.E.E. (1979) and Ph.D. (1984) U.Pennsylvania	**Functional** query languages for databases
Assoc. Prof., MIT EECS, 1984-1991 Researcher, Digital Equipment Corp. (1991-2000)	Dataflow/multithreaded computer **architectures** for HPC (High Performance Computing) **Functional** programming languages for HPC Contributor to first **Haskell** definition (1990-1992)
Project lead, Sandburst Corp. (2000-present) Co-founder and CTO, Bluespec, Inc.	Bluespec HL-HDL (High-Level **Hardware Design** Language) for complex **hardware designs/architectures**

An SoC is best viewed as a concurrent, distributed system

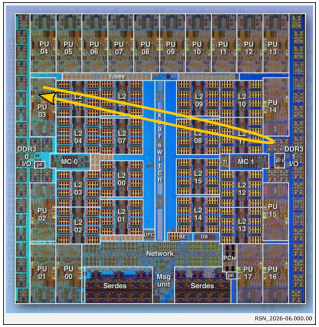
PDF of slides
Source codes of examples
`https://github.com/rsnikhil/Talk_Bring_UNITY_to_SW_HW_Design`

Concurrency and Distribution even within a chip

HW units on a chip (CPUs, Accels, memory subsystems) may be mere millimeters apart ... but:

There can be a *huge ratio* between the delay to access adjacent data *vs.* data across a chip (single clock cycle *vs.* hundreds of clock cycles).
(And even more orders of magnitude to go across chips and boards.)

- It is infeasible to have a high-speed global clock in exact synchrony across a chip.
  $\Longrightarrow$ HW units across chip should be viewed as **independently-asynchronous concurrent processes**.
- Accessing data across a chip must usually be *split-phase*: a request message and, after an unpredictable delay, a response message.
  $\Longrightarrow$ HW units across chip should be viewed as a **distributed system**.
- Accessing data across a chip usually involves multi-step communication through shared paths **NoCs (networks-on-a-chip)/interconnects**, with familiar concerns of serialization, congestion, deadlock, ordering, *etc.*



CPUs/Accels/Systems design should focus on concurrency, distribution

Unfortunately, traditional training in HW-design is bottom-up, inculcating a mindset of clocked, synchronous, digital circuits.

That may have been fine for SSI and MSI, but not for LSI or VLSI.

Instead, complex HW system design should focus on concurrency and distribution, treating clocked, synchronous, digital circuits as limited only to the internals of small, local, modules.

4th USENIX Workshop on Hot Topics in Parallelism (HotPar 12), Berkeley, CA, 2012 (deprecating "locks", "semaphores", serializability and other sequential thinking)

Calls to Action

Accept into your life: "Parallel" is the norm, not radical exception

"Simultaneous-action-at-a-distance" is a bad habit:

In summary ...

Complex Hardware (systems-on-a-chip–CPUs, multicore CPUs, HW-Accels, memory systems, interconnects, ...) should be *specified, modeled, designed, programmed and verified as asynchronous, concurrent, distributed systems*.

Traditional clocked, synchronous, digital circuit design methodology, if used at all, should be confined, to local detail inside small sub-units.

The main traditional HDLs (Hardware Design Languages)—Verilog, SystemVerilog, VHDL— and indeed many newer HDLs are focused on the latter, not on the former.

For SoC-level hardware design, we should move to a higher level of abtraction, away from clocked digital circuits towards graphs of processes:

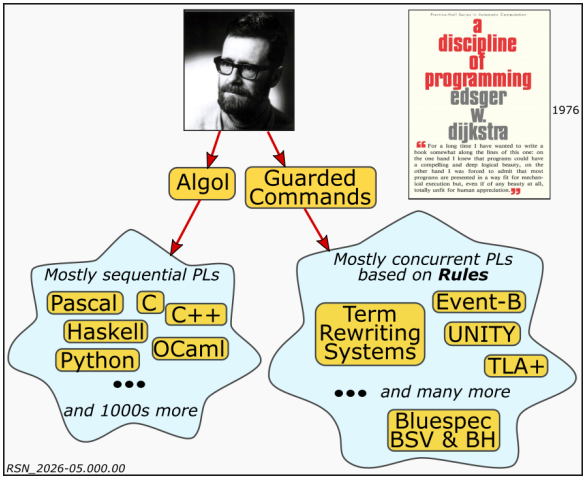
Parallel, asynchronous, concurrent, distributed, overlapped, latency-tolerant, split-phase, message-passing, reorder-tolerant, elastic-pipelines, dataflow, ...

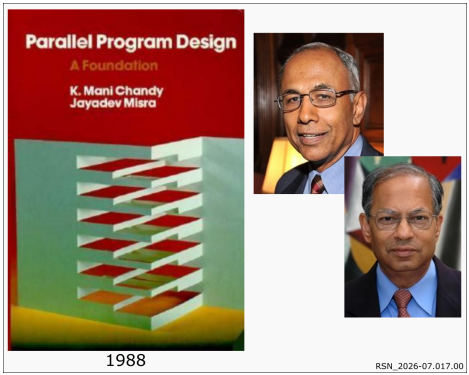
Dear Software Engineers! <u>You</u> are experts on such programming models!
<u>You</u> can apply them to HW design!

How should we program concurrent, distributed systems?

PDF of slides
Source codes of examples
`https://github.com/rsnikhil/Talk_Bring_UNITY_to_SW_HW_Design`

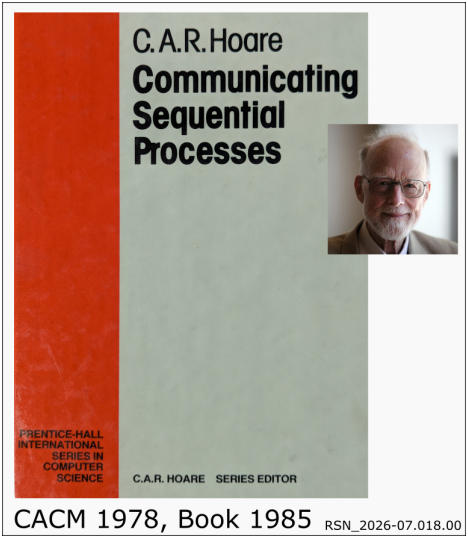
A good programming model for fundamentally *concurrent* systems





- UNITY programming language
- ... with a proof methodology.

A good programming model for fundamentally *distributed* systems



CACM 1978, Book 1985 RSN_2026-07.018.00

- Collection of *processes* (baseline sequential but, recursively, sub-collection of concurrent processes)
- Communicating bilaterally over declared *unidirectional channels*

  Bidirectional communication can be modeled with a pair of unidirectional channels.
- Default synchronous communication (sender and receiver must both be ready to act)

  Asynchronous communication with latency between processes A and B can be modeled with an intermediate process AB containing a bounded or unbounded **FIFO buffer**.
- ... with a proof methodology.

An example: Vector Addition (VAdd)

PDF of slides
Source codes of examples
`https://github.com/rsnikhil/Talk_Bring_UNITY_to_SW_HW_Design`

An example: Vector Addition (VAdd)

In the next few slides, with concrete code for a concrete example, we'll see how a Guarded-Command language (**BSV**) can be used to:

- to express a HW version of a software computation (VAdd), and offload it from a CPU;
- to *refine* the computation graph to admit more concurrency, for better performance;

Hopefully the **BSV** code we show for hardware is easily understandable by a software engineer.

(Incidentally, **BSV** was used to design all the hardware in these examples, including CPUs and memory system.)

An example: Vector Addition (VAdd)

Simple, familiar C code to add two vectors:

```
int vA[n], vB[n], vC[n]);

... // initialize vA and vB

for (int j = 0; j < n; j++)
    vC [j] = vA [j] + vB [j];

printf ("Done\n");

... // consume vC
```

This an $O(n)$ algorithm.

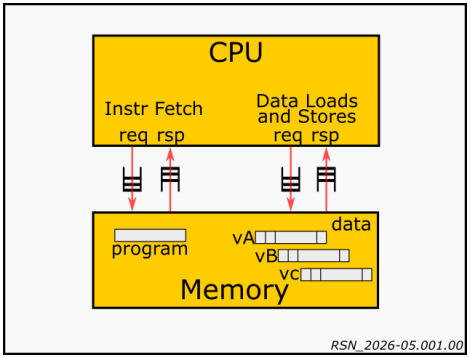
In complexity analysis and in modular software design we often focus on "the computation" (here, the "+" operation).

But when designing a hardware implementation, the engineering focus is on `vA[j]`, `vB[j]`, and `vC[j]`= ...
(loading and storing array elements from/to memory).

... because they involve *communication*, which is often vastly more expensive.

Baseline VAdd: traditional C/Asm program (no HW accel)

Let's Compile and Execute the Program on a CPU + Memory.



- CPU 1: our own **Drum** CPU (RISC-V RV32I, not pipelined)
- CPU 2: our own **Fife** CPU (RISC-V RV32I, 6-stage in-order pipeline)
- Coded in **BSV**
- Compiled to Verilog, which can be taken to FPGA or ASIC
- or can be run (as here) in Verilog simulation

BSV, the high-level hardware design language (HL-HDL) for our examples

BSV is similar to Guarded Commands [Dijkstra 1976], UNITY [Chandy+Misra 1988]:

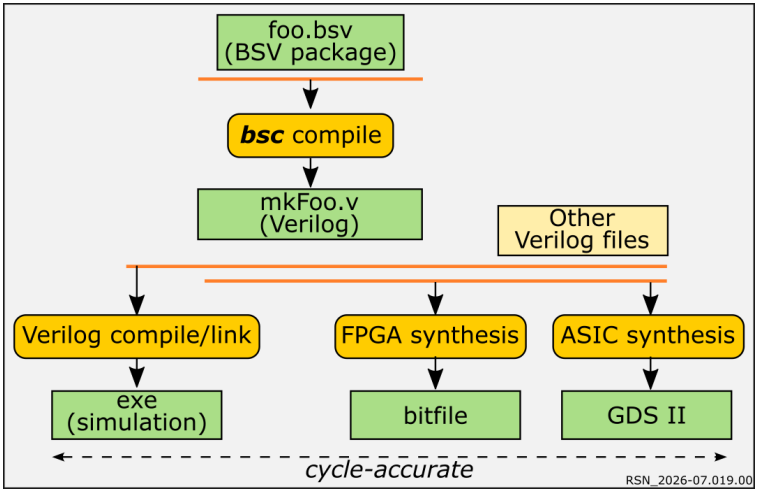
- All behavior is expressed in terms of *Rules* (Guarded Commands).
  A Rule is a condition-action pair.
- A rule-body can be a complex action, composed (recursively) of simpler actions.
- Behavior is simple, concurrent and non-deterministic: Repeatedly execute any rule whose condition is true, in any order, performing the action(s) in the rule body.
  (or, concurrently execute enabled rules **atomically**)

- Rules can express arbitrary concurrency patterns.
- Often, rules are used for *structured* concurrent processes (sequential, parallel fork-join, loops, if-then-else, *etc.*.).

  For these, **BSV** has convenient syntactic abstractions ("`StmtFSM`")..
- In our VAdd example, we only need structured processes (the `Stmt` construct).

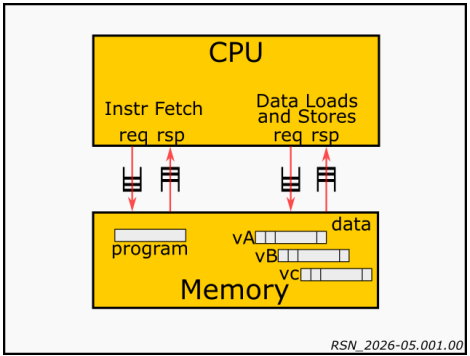
BSV is compiled to hardware *via* Verilog

... which can be simulated or taken to silicon (FPGA/ASIC).



Baseline VAdd: traditional C/Asm program (no HW accel)

Let's Compile and Execute the Program on a CPU + Memory.



- CPU 1: our own **Drum** CPU (RISC-V RV32I, not pipelined)
- CPU 2: our own **Fife** CPU (RISC-V RV32I, 6-stage in-order pipeline)
- Coded in **BSV**
- Compiled to Verilog, which can be taken to FPGA or ASIC
- or can be run (as here) in Verilog simulation

Let's run it **(optional demo)**:

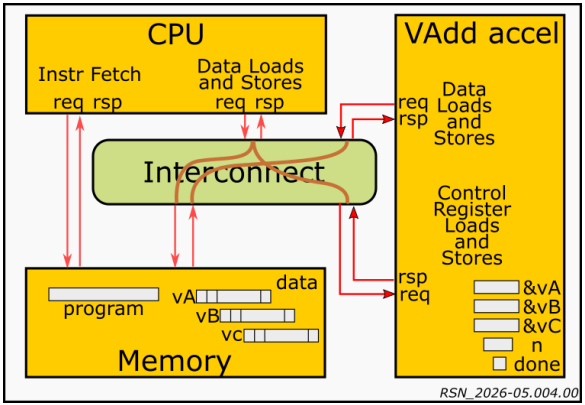
Vector size N	100	200
Drum cycles	6,376	12,676
Fife cycles	2,849	5,649

(Absolute # of cycles not important; just compare it to versions that follow.)

(Cycle-counts would improve with better branch prediction; bypassing; super-scalarity; out-of-order, but not by orders of magnitude.)

(In some systems Instruction-Fetch and Data-Load/Store share paths to memory, and can therefore run slower.)

Let's move Vector-Add to a hardware accelerator: version 1



Remaining program on CPU:
(actually in C but shown here in **BSV**, to illustrate that **BSV** can express sequential processes as a special case of concurrency)

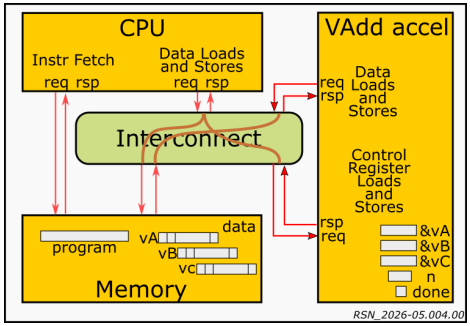
```
Reg #(int) rg_status <- mkReg (0);
write_req_rsp (Accel, 0, 1);
read_req_rsp (Accel, 1, rg_status);
```

Program on Accel:
(version 1: direct translitertation of original C/Asm code)

```
Reg #(int) j  <- mkRegU;
Reg #(int) aj <- mkRegU;
Reg #(int) bj <- mkRegU;
Reg #(int) cj <- mkRegU;

Stmt stmt_v1 =
seq
   for (j <= 0; j < rg_N; j <= j + 1)
   seq
      read_req_rsp  (VA, j, aj);
      read_req_rsp  (VB, j, bj);
      write_req_rsp (VC, j, aj + bj);
   endseq
endseq;
```

Let's run our Vector-Add hardware accelerator, version 1



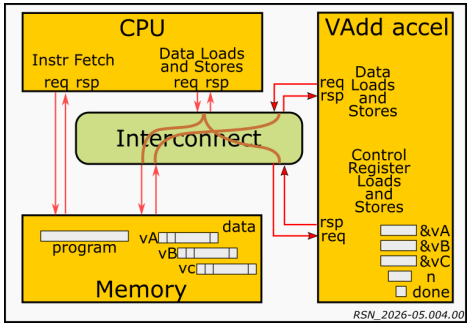
Vector size N	100
Drum cycles	6,376
Fife cycles	2,849
Fife + accel, v1	1,643
Fife + accel, v2	1,243
Fife + accel, v3	793
Fife + accel, v4	452

Optional demo

Compile full system (CPU, Memory System, Interconnect, Accelerator) to Verilog and run it in a Verilog simulator.

(Absolute # of cycles not important; just compare it to versions that follow.)

HW refinement: same code, expose split-phase request/response



Vector size N	100
Drum cycles	6,376
Fife cycles	2,849
Fife + accel, v1	1,643
Fife + accel, v2	1,243
Fife + accel, v3	793
Fife + accel, v4	452

Program on Accel:
(version 1b: just expose split-phase req/rsp)

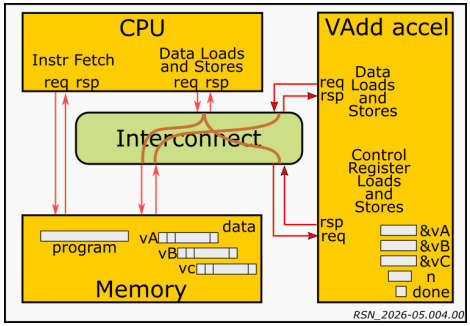
```
Stmt stmt_v1a =
seq
   for (j <= 0; j < rg_N; j <= j + 1)
   seq
      read_req (VA, j);
      read_rsp (VA, j, aj);

      read_req (VB, j);
      read_rsp (VB, j, bj);

      write_req (VC, j, aj + bj);
      write_rsp (VC, j);
   endseq
endseq;
```

(This is essentially the same program as Version 1; no change in cycle count.)

HW refinement: reorder read requests and responses: version 2



Vector size N	100
Drum cycles	6,376
Fife cycles	2,849
Fife + accel, v1	1,643
Fife + accel, v2	1,243
Fife + accel, v3	793
Fife + accel, v4	452

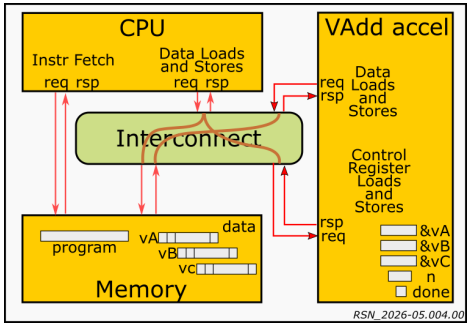
Program on Accel:
(version 2: reorder read req/rsp)

```
Stmt stmt_v2 =
seq
   for (j <= 0; j < rg_N; j <= j + 1)
   seq
      read_req (VA, j);    // launch
      read_req (VB, j);    // launch

      read_rsp (VA, j, aj);
      read_rsp (VB, j, bj);

      write_req (VC, j, aj + bj);
      write_rsp (VC, j);
   endseq
endseq;
```

HW refinement: loop unroll and reorder: version 3



Vector size N	100
Drum cycles	6,376
Fife cycles	2,849
Fife + accel, v1	1,643
Fife + accel, v2	1,243
Fife + accel, v3	793
Fife + accel, v4	452

Program on Accel:
(version 3: loop unroll and reorder)

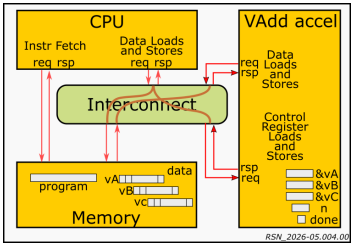
```
Stmt stmt_v3 =
seq
   for (j <= 0; j < rg_N; j <= j + 2)
   seq
      read_req (VA, j);
      read_req (VB, j);
      read_req (VA, j+1);
      read_req (VB, j+1);

      read_rsp  (VA, j, aj);
      read_rsp  (VB, j, bj);
      write_req (VC, j, aj + bj);

      read_rsp  (VA, j+1, aj);
      read_rsp  (VB, j+1, bj);
      write_req (VC, j+1, aj + bj);

      write_rsp (VC, j);
      write_rsp (VC, j+1);
   endseq
endseq;
```

HW refinement: concurrent pipeline: version 4



Vector size N	100
Drum cycles	6,376
Fife cycles	2,849
Fife + accel, v1	1,643
Fife + accel, v2	1,243
Fife + accel, v3	793
Fife + accel, v4	452

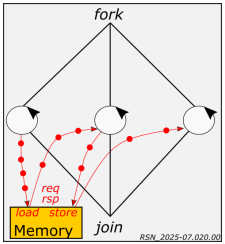
Version 4: concurrent, pipelined, using par/endpar (fork-join parallelism)

```
Reg #(int) j2 <- mkRegU;  Reg #(int) j3 <- mkRegU;

Stmt stmt_v4 =
par
   for (j <= 0; j < vsize; j <= j + 1) seq
      read_req (VA, j);
      read_req (VB, j);
   endseq

   for (j2 <= 0; j2 < vsize; j2 <= j2 + 1) seq
      read_rsp  (VA, j, aj);
      read_rsp  (VB, j, bj);
      write_req (VC, j, aj + bj);
   endseq

   for (j3 <= 0; j3 < vsize; j3 <= j3 + 1)
      write_rsp (VC, j);
endpar;
```



Caution! Potential deadlock due to head-of-line blocking!

Three concurrent sequential loops:

The 1st for-loop streams out load-requests.
The 2nd for-loop streams in load-responses, performs additions, and streams out store-requests.
The 3rd for-loop streams in write-responses.

Conclusion and Resources

PDF of slides
Source codes of examples
`https://github.com/rsnikhil/Talk_Bring_UNITY_to_SW_HW_Design`

Summary of Results

Substantial performance gains can be obtained by:

- Partitioning a computation into SW and direct-HW-execution parts. Direct HW execution removes a layer of interpretation.
- Micro-architectural refinements of the HW implementation, to further improve performance through concurrency and pipelining.

Vector size N	100
Drum cycles	6,376
Fife cycles	2,849
Fife + accel, v1	1,643
Fife + accel, v2	1,243
Fife + accel, v3	793
Fife + accel, v4	452

A computation model and language (like **BSV** focused on asynchronous concurrency and distribution (Guarded Commands, unidirectional FIFO-like communication channels) facilitates both activities.

Resources

These slides (PDF), and source codes for all the examples in this talk (**BSV** source codes, build scripts), including the Drum and Fife RISC-V CPUs.

`https://github.com/rsnikhil/Talk_Bring_UNITY_to_SW_HW_Design`

BSV and **BH** language documentation
bsc compiler for **BSV** and **BH**, libraries
	`https://github.com/B-Lang-org/bsc`
	`https://github.com/B-Lang-org/bsc-contrib`
	`https://github.com/BSVLang/Main`

Discussion forums	`https://groups.io/g/b-lang-discuss` `https://groups.io/g/b-lang-announce/topics`
Textbook to learn Bluespec **BSV** and to learn to design a pipelined RISC-V CPU Includes full **BSV** source code for two RISC-V CPUs: **Drum**: Reference simple, unpipelined RV32I **Fife**: 6+ stage pipelined RV32I	`https://github.com/rsnikhil/Learn_Bluespec_and_RISCV_Design`

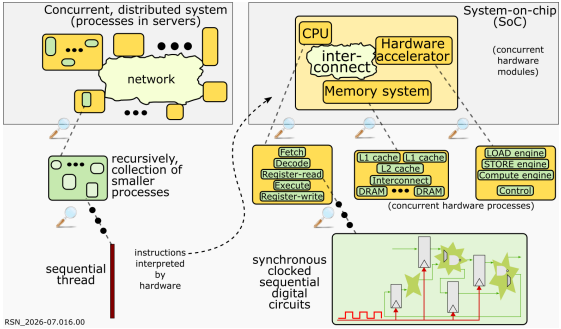
Conclusion

For chip-scale hardware systems (SoCs), traditional synchronous, clocked, sequential digital circuit design should be relegated to the bottom-most (most local) layer.

All but most the lowest layers of an SoC should be programmed as an asynchronous, latency-tolerant, concurrent, distributed system.

Software Engineers should embrace hardware design!

- You can shape and teach HW design methodology, with programming models familiar to you (asynchronous, concurrent, distributed system)!
- You could roll your own hardware accelerators!
  - Everybody needs accelerators.
  - Capable FPGAs are now affordable and accessible.
  - You have the programming tools.



Physical span: office, rack, data center, inter-city, inter-continent

Physical span: centimeter

Q & A?

Extras

Extending/evolving this accelerator

In the accelerator structure here, we can easily replace the computation with other memory-to-memory computations:

- Matrix Multiplication and other Linear Algebra
- Sorting
- Encryption/Decription
- Image filters

Many topics we have elided here:

- Interconnects (buses, rings, grids, multi-stage recursive networks, cross-bars ...)
- Multiple paths to/from memory
- Memory banks
- Detecting completion: Polling vs. Interrupts
- Batch multiple reads ("cache line")
- Batch multiple writes (write-merge/coalesce, "cache line")
- Cache coherence with CPU
- Virtual Memory (MMUs)
- Shared accelerators and security/protection
- Context-switching

Can the original C code be *compiled* to the final hardware?

- Compiling this particular C example (vector-add) to hardware is possible. There are tools, even commercial products, that purport to do that, under the rubric of "HLS" (High Level Synthesis).

- But, IMHO, HLS methodology does not scale to more complex designs (complex control structure, variable memory latencies, non-affine vector indexes, sparse matrices, ...).

  Whereas *Hardware Design Languages (HDLs)* like Verilog, VHDL and SystemVerilog, and *High-Level HDLs (HLHDLs)* like Bluespec **BSV**/**BH** are "universal"—suitable for designing all digital hardware.

- The computation semantics of software programming languages are an abstraction of their eventual target (machine code).

  Within this family, there are lower-level languages and higher-level languages (Assembly, C, C++, Rust, Python, Kotlin, Haskell, OCaml, ...).

- The computation semantics of HDLs are an abstraction of their eventual target: digital logic, with fine-grained concurrency and fine-grained cooperating FSMs.

  Within this family there are lower-level languages and higher-level languages (Verilog, VHDL, SystemVerilog, Chisel, Bluespec **BSV**/**BH**, ...)